PLP - Segundo Recuperatorio - 1do cuatrimestre de 2024

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
117.	64/22	Kerbs Octavio	B-	B-	B-	A

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Resolución

a) Representar en forma clausal las siguientes fórmulas de lógica de primer orden:

1. $\exists C.\forall X.(X \in a \Rightarrow X \leq C)$
   Existe una cota superior para el conjunto a.
2. $\forall X.\forall Y.(X \leq Y \Rightarrow n(Y) \leq n(X))$
   Si $X \leq Y$, entonces $-Y \leq -X$.
3. $\forall X.(X \in a \Leftrightarrow n(X) \in b)$
   X pertenece al conjunto a si y solo si -X pertenece al conjunto b.

b) Utilizando resolución, determinar si existe una cota inferior para el conjunto b: $\exists C.\forall X.(n(X) \in b \Rightarrow C \leq n(X))$.

c) La resolución utilizada en el punto anterior, ¿fue SLD? Justificar.

Ejercicio 2 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`.

a) Implementar el predicado `generarCapicúas(-L)`, que genera todas las listas capicúas de números naturales (sin incluir al cero). Por ejemplo:

```
?- generarCapicuas(L).
L = [1] ;
L = [2] ;
...
L = [1,3,1] ;
...
L = [2,3,3,2];
...
```

b) El predicado anterior ¿es reversible? Justificar.

c) Definir el predicado `tokenizar(+D,+F,-T)`, que es verdadero cuando `D` representa un diccionario de palabras conocidas (lista de listas de letras), `F` es una lista de letras sin espacios, y `T` es la lista de palabras en la que se puede partir la frase. El predicado debe generar todas las posibles maneras de dividir la frase en palabras conocidas. Por ejemplo:

```
?- tokenizar([[a],[a,b],[b,a],[c]], [a,b,a,c,a,b,a], T).
T = [[a],[b,a],[c],[a],[b,a]];
T = [[a],[b,a],[c],[a,b],[a]];
T = [[a,b],[a],[c],[a],[b,a]];
T = [[a,b],[a],[c],[a,b],[a]];
false.
```

d) Definir el predicado `mayorCantPalabras(+D,+F,-T)` que es verdadero cuando `T` es una tokenización de la frase `F` con el diccionario `D` que tiene la mayor cantidad de palabras. En este inciso no está permitido utilizar estructuras auxiliares.

Ejercicio 3 - Objetos y Deducción Natural

a) Considere las siguientes clases:

```
Object subclass: #A

    m1
    ^[self eval].

    eval
    ^self value.

    value
    ^0.
```

```
A subclass: #B

    m2
    ^[super eval].

    m3
    ^self.

    value
    ^1.

    eval
    ^2.
```

Para cada una de las siguientes expresiones, hacer una tabla donde se indique, en orden, cada mensaje que se envía, qué objeto lo recibe, en qué clase está el método respectivo, y cuál es el resultado final de cada colaboración:

I) `B new m1 value`

II) `B new m3 m2 value`

b) Implementar un *closure* que tome como parámetro una colección y devuelva los elementos de la colección que saben responder al mensaje `#ptff`.
Sugerencia: pueden utilizar el mensaje `#respondsTo:`, que toma un mensaje y devuelve si el objeto sabe responderlo.

c) Demostrar en deducción natural que vale la siguiente fórmula, sin utilizar principios de razonamiento clásicos:

$$\exists X.P(X) \vee \forall Y.Q(Y) \implies \exists X.(Q(X) \vee P(X))$$

(H1) ①

B-

I) $\exists C. \forall x. (x \in a \Rightarrow x \leq C)$

$\equiv \exists C. \forall x. (\neg(x \in a) \vee (x \leq C))$.

① $\overset{\text{SKOLEM}}{\equiv} \{\neg(x_1 \in a), (x_1 \leq C)\}$ ✓

II) $\forall x. \forall y. (x \leq y \Rightarrow n(y) \leq n(x))$.

$\equiv \forall x. \forall y. (\neg(x \leq y) \vee (n(y) \leq n(x)))$.

② $\equiv \{\neg(x_2 \leq y_2), (n(y_2) \leq n(x_2))\}$. ✓

III) $\forall x. (x \in a \leftrightarrow n(x) \in b)$.

$\equiv \forall x. ((x \in a) \rightarrow (n(x) \in b)) \wedge ((n(x) \in b) \rightarrow (x \in a)))$.

$\equiv \forall x. ((\neg(x \in a) \vee (n(x) \in b)) \wedge (\neg(n(x) \in b) \vee (x \in a)))$.

③

$\equiv \{\{\neg(x_3 \in a), (n(x_3) \in b)\}, \{\neg(n(x_4) \in b), (x_4 \in a)\}\}$. ✓ ✓

④

b) $\neg(\exists C. \forall x. (n(x) \in b \rightarrow C \leq n(x)))$.

$\equiv \forall C. \exists x. ((n(x) \in b) \wedge \neg(C \leq n(x)))$.

$\overset{\text{SKOLEM}}{\equiv} \{\{(n(f(C_5)) \in b\}, \{\neg(C_6 \leq n(f(C_6)))\}\}$

⑤ ✓ ⑥ ✓

⑦. $\{\neg(f(n(y_2)) \leq y_2)\}$ de 6 y 2.

$\sigma_1 = \{C_6 \leftarrow n(y_2), x_2 \leftarrow f(C_6)\}$. ✓

⑧. $\{\neg(f(n(c)) \in a)\}$ de 7 y 1

$\sigma_2 = \{x_1 \leftarrow f(n(y_2)), y_2 \leftarrow c\}$. ✓

(9) $\{\neg(n(f(n(c))) \in b\}$ de 8 y 4.

$\sigma_3 = \{X_4 \leftarrow f(n(c))\}$.

(10) $\{\square\}$ de 9 y 5.

$\sigma_4 = \{X_5 \leftarrow n(c)\}$

→ La resolución es SLD porque son todas cláusulas de horn, fue lineal, comenzamos por una cláusula objetivo. (6).

Además utiliza resolución binaria en todos los casos.

Nombre: Octavio Kenks Hoja Nº 2

69/22

(2)

a) generarCapicúas(L) :- desde (1, N),
listaDeN(N, L),
capicúa(L).

• listaDeN(0, []).
listaDeN(N, L) :-
N > 0,
between(1, N, N2),
N3 is N - N2,
listaDeN(N3, L1),
append([N2], L1, L).

• capicúa([]). %Asumo que la lista vacía es capicúa.

Esta corrección está mal, capicúa está bien implementado (lo hablé con el corrector)

Falta caso base

capicúa([_]).
capicúa([X]).
capicúa(L) :-
append([X], Xs, L),
capicúa(Ys).

• desde(X, X).
desde(X, Y) :- N is X+1, desde(N, Y).

b) Con reversible supongo que se refiere a que si yo le doy un arreglo instanciado, me dev. true si es capicúa y false en caso contrario.

Supongamos que la lista es L = [2]. (es capicúa). desde (1, N) instancia a N en 1. Luego se resuelve listoDeN. Se instancia N2 = 1, se instancia N3 = 0. Se instancia L1 = [], y por último el append consulta si [1|[]] = [2]. Lo cual es falso.

~~-> No es reversible. Porque L = [2] es capicúa pero generar Capicuas no encontró una.~~

-> Estoy pensando que si se backtrackea al "desde", cuando N = 2, va a encontrar una instancia donde [2|[]] = L = [2] y es capicúa pero luego va a hacer consultas infinitas. Nos va a dar un true y se va a colgar.

-> Una posible solución sería definir otra regla para generar Capicúas donde chequee si L viene instanciado con naVar(L). Si viene instanciado solo chequeamos si es capicúa. Caso contrario, lo que definí yo (agregando Var(L)).

NO ESCRIBAS COSAS QUE NO TE PEDIMOS, SI ESTÁ BIEN NO SUMA, SI ESTÁ MAL SI RESTA.

c)

```
• tokenizar(_, [], []).

tokenizar(D, F, [P|Xs]) :-
    member(P, D),
    append(P, Ys, F),
    tokenizar(D, Ys, Xs).
```

d) • mayorContPalabras(D, F, T) :-

```
    tokenizarLong(D, F, T, N),
    not((tokenizarLong(D, F, _, N2), N2 > N))
```

• tokenizarLong(D, F, T, N) :-

```
    tokenizar(D, F, T),
    length(T, N).
```

③. a)

B-

mensaje	receptor	ubicación	resultado.
new	B	Object Class	aB.
m1	aB	A	[self eval]
value	[self eval]	BlockClosure	2
eval	aB	B	2.
new	B	Object Class	anothenB
m3	anothenB	B	anothenB.
m2	anothenB	B	[super eval]
value	[super eval]	BlockClosure	1.
eval	anotenB	A	1
value	anothenB	B	1

—> Resultado final

I) 2

I) 1.

Esta return la vea del método

b). [:xs |

^ (xs select: [:x | x respondsTo: #pctt])

].

—> no está especificado el formato que toma ~~de pctt~~. Lo paso como dice el enunciado. "respondsTo:".

Nombre: Octavio Kanbs Hoja Nº 5

64/22

$\star\ X \notin fv(\gamma, \exists x.(Q(x) \vee P(x)))$

$$
\dfrac{
\dfrac{}{\Gamma \vdash \exists x.P(x) \vee \forall y.Q(y)}\,Ax
\qquad
\dfrac{
\dfrac{}{\gamma \vdash \exists x.P(x)}\,Ax
\qquad
\dfrac{
\dfrac{\dfrac{}{\gamma, P(x) \vdash P(x)}\,Ax}{\gamma, P(x) \vdash Q(x) \vee P(x)\ \{X := x\}}\,\vee i_2
\qquad
\dfrac{\dfrac{\dfrac{}{\psi \vdash \forall y.Q(y)}\,Ax}{\psi \vdash Q(y)\ \{y := y\}}\,\forall e}{\psi \vdash Q(y) \vee P(y)\ \{X := y\}}\,\vee i_1
}{\gamma, P(x) \vdash \exists x.(Q(x) \vee P(x))}\,\exists i
}{\gamma : \Gamma, \exists x.P(x) \vdash \exists x.(Q(x) \vee P(x))}\,\exists e
}{\Gamma : \exists x.P(x) \vee \forall y.Q(y) \vdash \exists x.(Q(x) \vee P(x))}\,\vee e
}{\emptyset \vdash \exists x.P(x) \vee \forall y.Q(y) \Rightarrow \exists x.(Q(x) \vee P(x))}\,\rightarrow i
$$